

Lab 4: Doubly Linked Lists

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Sentinel nodes, bidirectional linking, addFirst/addLast/remove

Objectives

- Understand why sentinel (header/trailer) nodes simplify boundary conditions.
- Implement a generic `DoublyLinkedList<E>` with `prev` and `next` pointers.
- Implement `addFirst`, `addLast`, and `remove(Node)` in $O(1)$ time.
- Compare the trade-offs of singly vs. doubly linked lists.

Quick Reference

Node class.

```
private static class Node<E> {
    E element;
    Node<E> prev, next;
    Node(E e, Node<E> p, Node<E> n) { element = e; prev = p; next = n; }
}
```

Sentinel setup.

```
header = new Node<>(null, null, null);
trailer = new Node<>(null, header, null);
header.next = trailer;
```

addBetween (private helper):

```
Node<E> node = new Node<>(e, predecessor, successor);
predecessor.next = node;
successor.prev = node;
size++;
```

remove(Node node):

```
Node<E> p = node.prev, s = node.next;
p.next = s; s.prev = p;
size--;
return node.element;
```

Quick Experiments (Try It)

Try It 1: Sentinels eliminate special cases

Add a print inside `addBetween` to confirm the same code path handles both the empty and non-empty cases. How many `if`-statements does `addBetween` need?

Try It 2: Backward traversal

Traverse your list from `trailer.prev` back to `header.next`. Verify the output is the reverse of the forward traversal.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Implement `DoublyLinkedList<E>` (50 min)

Build the class using header/trailer sentinels:

- `int size()`, `boolean isEmpty()`.
- `E first()`, `E last()` — null if empty.
- `void addFirst(E e)`, `void addLast(E e)`.
- `E removeFirst()`, `E removeLast()` — both $O(1)$.
- `String toString()` — e.g. `[A <-> B <-> C]`.

Use private `addBetween` and `remove(Node)` helpers. Test with at least 10 operations including removes from both ends.

In-Lab Exercise 2: Add at Position (25 min)

Add `void add(int index, E e)` that inserts at zero-based position `index`. Traverse to node at position `index`, then call `addBetween`. What is the time complexity? When is this worse than an array?

In-Lab Exercise 3: Deque Adapter (25 min)

Wrap `DoublyLinkedList<E>` in a `LinkedDeque<E>` with: `addFirst`, `addLast`, `removeFirst`, `removeLast`, `peekFirst`, `peekLast`, `size`, `isEmpty` — all $O(1)$.

In-Lab Exercise 4: Palindrome Checker (20 min)

Using `LinkedDeque<Character>`, write `boolean isPalindrome(String s)`: add each character to the deque, then repeatedly remove from both ends and compare. Test with "racecar", "hello", and "madam".

AI Collaboration

Suggested prompts:

- “Explain how sentinel nodes eliminate edge-case checks in a doubly linked list.”
- “Why is `removeLast` $O(1)$ in a doubly linked list but $O(n)$ in a singly linked list?”

Verify: Ask the AI to draw pointer changes for `addFirst` and `removeLast` on a one-real-node list. Run those operations and confirm the sentinel pointers match.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Pointer trace with sentinels

Draw the doubly linked list (including sentinels H and T) after each step. Show all `prev/next` arrows.

```
DoublyLinkedList<String> L = new DoublyLinkedList<>();
L.addFirst("B"); L.addFirst("A"); L.addLast("C"); L.removeFirst(); L.removeLast();
```

Write Code on Paper 1: Reverse in-place

Write `void reverse()` for `DoublyLinkedList`: swap `prev` and `next` for every node including sentinels. No new nodes created.

Draw on Paper 1: Singly vs. doubly linked list

Draw side-by-side memory diagrams of a three-element singly linked list and a three-element doubly linked list (with sentinels). Label every pointer field. Circle the extra fields in the doubly linked version and state the per-node memory overhead.

Submission. Save files as `lab4_ex*.java` and submit on the course site.